Lab: Build a Flight Table with Airline Names

Run it from the folder that contains this notebook.

Part: Download data and build working table

1. Install libraries from requirements.txt

In [1]:
```
%pip install -r requirements.txt
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas==2.3.3 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-p
ackages (from -r requirements.txt (line 1)) (2.3.3)
Requirement already satisfied: matplotlib==3.9.4 in /Users/nguyentritin/Library/Python/3.9/lib/python/si
te-packages (from -r requirements.txt (line 2)) (3.9.4)
Requirement already satisfied: nycflights13==0.0.3 in /Users/nguyentritin/Library/Python/3.9/lib/python/
site-packages (from -r requirements.txt (line 3)) (0.0.3)
Requirement already satisfied: tzdata>=2022.7 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-
packages (from pandas==2.3.3->-r requirements.txt (line 1)) (2026.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/nguyentritin/Library/Python/3.9/lib/pyth
on/site-packages (from pandas==2.3.3->-r requirements.txt (line 1)) (2.9.0.post0)
Requirement already satisfied: numpy>=1.22.4 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-p
ackages (from pandas==2.3.3->-r requirements.txt (line 1)) (2.0.2)
Requirement already satisfied: pytz>=2020.1 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-pa
ckages (from pandas==2.3.3->-r requirements.txt (line 1)) (2026.5)
Requirement already satisfied: contourpy>=1.0.1 in /Users/nguyentritin/Library/Python/3.9/lib/python/sit
e-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (1.3.0)
Requirement already satisfied: packaging>=20.0 in /Users/nguyentritin/Library/Python/3.9/lib/python/site
-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (26.3)
Requirement already satisfied: importlib-resources>=3.2.0 in /Users/nguyentritin/Library/Python/3.9/lib/
python/site-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (6.5.2)
Requirement already satisfied: pillow>=8 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-packa
ges (from matplotlib==3.9.4->-r requirements.txt (line 2)) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /Users/nguyentritin/Library/Python/3.9/lib/python/sit
e-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (3.3.3)
Requirement already satisfied: fonttools>=4.22.0 in /Users/nguyentritin/Library/Python/3.9/lib/python/si
te-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (4.60.2)
Requirement already satisfied: kiwisolver>=1.3.1 in /Users/nguyentritin/Library/Python/3.9/lib/python/si
te-packages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (1.4.7)
Requirement already satisfied: cycler>=0.10 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-pa
ckages (from matplotlib==3.9.4->-r requirements.txt (line 2)) (0.12.1)
Requirement already satisfied: zipp>=3.1.0 in /Users/nguyentritin/Library/Python/3.9/lib/python/site-pac
kages (from importlib-resources>=3.2.0->matplotlib==3.9.4->-r requirements.txt (line 2)) (3.23.1)
Requirement already satisfied: six>=1.5 in /Library/Developer/CommandLineTools/Library/Frameworks/Python
3.framework/Versions/3.9/lib/python3.9/site-packages (from python-dateutil>=2.8.2->pandas==2.3.3->-r req
uirements.txt (line 1)) (1.15.0)
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.
You should consider upgrading via the '/Library/Developer/CommandLineTools/usr/bin/python3 -m pip instal
l --upgrade pip' command.
Note: you may need to restart the kernel to use updated packages.
```

2. Download the dataset into data/raw

In [2]:
```
from nycflights13 import flights, airlines


flights.to_csv("data/raw/flights.csv", index=False)
airlines.to_csv("data/raw/airlines.csv", index=False)
```

3. Set up libraries and file paths

In [3]:
```
from pathlib import Path
import pandas as pd

ROOT_DIR = Path.cwd()
FLIGHTS_PATH = ROOT_DIR / "data" / "raw" / "flights.csv"
AIRLINES_PATH = ROOT_DIR / "data" / "raw" / "airlines.csv"
OUTPUT_PATH = ROOT_DIR / "data" / "processed" / "flights_working.csv"
```

```
print(f"Lab directory: {ROOT_DIR}")
print(f"Found flights.csv: {FLIGHTS_PATH.is_file()}")
print(f"Found airlines.csv: {AIRLINES_PATH.is_file()}")
```

```
Lab directory: /Users/nguyentritin/introds-group09-flight-delays
Found flights.csv: True
Found airlines.csv: True
```

4. Load and preview the input data

In [4]:
```
if not FLIGHTS_PATH.is_file() or not AIRLINES_PATH.is_file():
    raise FileNotFoundError("Input CSV files were not found in data/raw.")

flights = pd.read_csv(FLIGHTS_PATH)
airlines = pd.read_csv(AIRLINES_PATH)

print(f"flights: {flights.shape[0]:,} rows, {flights.shape[1]} columns")
print(f"airlines: {airlines.shape[0]:,} rows, {airlines.shape[1]} columns")
display(flights.head(3))
display(airlines.head())
```

```
flights: 336,776 rows, 19 columns
airlines: 16 rows, 2 columns
```

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum
0	2013	1	1	517.0	515	2.0	830.0	819	11.0	UA	1545	N14228
1	2013	1	1	533.0	529	4.0	850.0	830	20.0	UA	1714	N24211
2	2013	1	1	542.0	540	2.0	923.0	850	33.0	AA	1141	N619AA

	carrier	name
0	9E	Endeavor Air Inc.
1	AA	American Airlines Inc.
2	AS	Alaska Airlines Inc.
3	B6	JetBlue Airways
4	DL	Delta Air Lines Inc.

5. Join airline names to each flight

Use a `left` join to keep every flight. Carrier codes are stripped of surrounding whitespace before joining; each airline carrier code must be unique, and every flight must match an airline.

In [5]:
```
if "carrier" not in flights.columns:
    raise ValueError("Flights data must contain a carrier column.")
if not {"carrier", "name"}.issubset(airlines.columns):
    raise ValueError("Airlines data must contain carrier and name columns.")
if "name" in flights.columns:
    raise ValueError("Flights data already contains a name column; check the input data.")

flights = flights.copy()
airlines = airlines[["carrier", "name"]].copy()
flights["carrier"] = flights["carrier"].astype("string").str.strip()
airlines["carrier"] = airlines["carrier"].astype("string").str.strip()
airlines["name"] = airlines["name"].astype("string").str.strip()

if flights["carrier"].isna().any() or flights["carrier"].eq("").any():
    raise ValueError("Flights data contains a missing or blank carrier code.")
if airlines[["carrier", "name"]].isna().any().any() or airlines[["carrier", "name"]].eq("").any().any(
):
    raise ValueError("Airlines data contains a missing or blank carrier code or airline name.")
if airlines["carrier"].duplicated().any():
    raise ValueError("Airlines data contains duplicate carrier codes.")

working_table = flights.merge(
    airlines,
    on="carrier",
```

```
    how="left",
    validate="many_to_one",
    indicator=True,
)
unmatched = sorted(working_table.loc[working_table["_merge"] == "left_only", "carrier"].unique())
if unmatched:
    raise ValueError(f"No airline found for these carrier codes: {unmatched}")
working_table = working_table.drop(columns="_merge")

assert len(working_table) == len(flights), "The join changed the number of flight rows."
print(f"Join successful: {len(working_table):,} flights, {len(working_table.columns)} columns.")
display(working_table[["carrier", "flight", "name"]].head())
```

```
Join successful: 336,776 flights, 20 columns.
```

	carrier	flight	name
0	UA	1545	United Air Lines Inc.
1	UA	1714	United Air Lines Inc.
2	AA	1141	American Airlines Inc.
3	B6	725	JetBlue Airways
4	DL	461	Delta Air Lines Inc.

6. Save the result

In [6]:
```
OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
working_table.to_csv(OUTPUT_PATH, index=False, lineterminator="\n")

print(f"Saved: {OUTPUT_PATH.relative_to(ROOT_DIR)}")
print(f"Size: {len(working_table):,} rows × {len(working_table.columns)} columns")
```

```
Saved: data/processed/flights_working.csv
Size: 336,776 rows × 20 columns
```

7. Reload and verify the result

Check that the saved file contains every flight, has no missing airline names, and maps each carrier to the correct airline.

In [7]:
```
saved_table = pd.read_csv(OUTPUT_PATH)
expected_names = airlines.set_index("carrier")["name"]
mapped_names = saved_table["carrier"].astype("string").str.strip().map(expected_names)

assert len(saved_table) == len(flights), "The output file does not contain every flight row."
assert saved_table["name"].notna().all(), "Some flights are missing an airline name."
assert saved_table["name"].astype("string").equals(mapped_names), "Airline names in the output file do
not match the lookup table."

print(f"Verification successful: {len(saved_table):,} rows; all airline names match.")
display(saved_table[["carrier", "flight", "name"]].head())
```

```
Verification successful: 336,776 rows; all airline names match.
```

	carrier	flight	name
0	UA	1545	United Air Lines Inc.
1	UA	1714	United Air Lines Inc.
2	AA	1141	American Airlines Inc.
3	B6	725	JetBlue Airways
4	DL	461	Delta Air Lines Inc.

Part: Data profile and cleaning log

In [8]:
```
import pandas as pd
import numpy as np

SRC = "data/processed/flights_working.csv"
df = pd.read_csv(SRC)
```

```
print(f"Rows: {len(df):,}   Columns: {df.shape[1]}")
```

```
Rows: 336,776   Columns: 20
```

8. Data profile (as loaded, before any cleaning)

In [9]:
```
def profile(d):
    n = len(d)
    out = []
    for c in d.columns:
        s = d[c]
        nn = int(s.notna().sum())
        if pd.api.types.is_numeric_dtype(s):
            rng = f"{s.min():,.0f} to {s.max():,.0f}"
        else:
            rng = f"{s.nunique():,} distinct"
        out.append((c, str(s.dtype), nn, n - nn, round((n - nn) / n * 100, 2), rng))
    return pd.DataFrame(out, columns=["Column", "Type", "Non-null", "Missing", "Missing %", "Range or distinct values"])

profile(df)
```

Out[9]:

	Column	Type	Non-null	Missing	Missing %	Range or distinct values
0	year	int64	336776	0	0.00	2,013 to 2,013
1	month	int64	336776	0	0.00	1 to 12
2	day	int64	336776	0	0.00	1 to 31
3	dep_time	float64	328521	8255	2.45	1 to 2,400
4	sched_dep_time	int64	336776	0	0.00	106 to 2,359
5	dep_delay	float64	328521	8255	2.45	-43 to 1,301
6	arr_time	float64	328063	8713	2.59	1 to 2,400
7	sched_arr_time	int64	336776	0	0.00	1 to 2,359
8	arr_delay	float64	327346	9430	2.80	-86 to 1,272
9	carrier	object	336776	0	0.00	16 distinct
10	flight	int64	336776	0	0.00	1 to 8,500
11	tailnum	object	334264	2512	0.75	4,043 distinct
12	origin	object	336776	0	0.00	3 distinct
13	dest	object	336776	0	0.00	105 distinct
14	air_time	float64	327346	9430	2.80	20 to 695
15	distance	int64	336776	0	0.00	17 to 4,983
16	hour	int64	336776	0	0.00	1 to 23
17	minute	int64	336776	0	0.00	0 to 59
18	time_hour	object	336776	0	0.00	6,936 distinct
19	name	object	336776	0	0.00	16 distinct

9. Cleaning log

Decision 1 — Merge check (airlines → flights on `carrier`)

In [10]:
```
print("carrier->name is 1:1:", df.groupby("carrier")["name"].nunique().max() == 1 and df.groupby("name")["carrier"].nunique().max() == 1)
print("Rows with no airline name:", df["name"].isna().sum())
print("Distinct carriers:", df["carrier"].nunique())
```

```
carrier->name is 1:1: True
Rows with no airline name: 0
Distinct carriers: 16
```

Decision 2 — Whole-number columns stored as float → nullable Int64

In [11]:
```
int_cols = ["dep_time", "dep_delay", "arr_time", "arr_delay", "air_time"]
assert all((df[c].dropna() % 1 == 0).all() for c in int_cols)   # nothing is lost by the conversion
df[int_cols] = df[int_cols].astype("Int64")
print(df[int_cols].dtypes)
```

```
dep_time     Int64
dep_delay    Int64
arr_time     Int64
arr_delay    Int64
air_time     Int64
dtype: object
```

Decision 3 — `time_hour`: text (UTC) → timezone-aware datetime in America/New_York

In [12]:
```
t = pd.to_datetime(df["time_hour"], utc=True).dt.tz_convert("America/New_York")
print("Unparseable:", t.isna().sum())
agree = ((t.dt.year == df["year"]) & (t.dt.month == df["month"]) & (t.dt.day == df["day"])).sum()
print("Local date agrees with year/month/day:", f"{agree:,} of {len(df):,}")
df["time_hour"] = t
```

```
Unparseable: 0
Local date agrees with year/month/day: 336,776 of 336,776
```

Decision 4 — Clock times: `2400` means midnight → recode to `0` in new columns

In [13]:
```
for c in ["dep_time", "arr_time"]:
    print(c, "== 2400:", int((df[c] == 2400).sum()))
    df[c + "_clock"] = df[c].mask(df[c] == 2400, 0)
print("Rows touched:", int(((df["dep_time"] == 2400) | (df["arr_time"] == 2400)).sum()))
print("sched_dep_time / sched_arr_time == 2400:", int((df["sched_dep_time"] == 2400).sum()), int((df["
sched_arr_time"] == 2400).sum()))
```

```
dep_time == 2400: 29
arr_time == 2400: 150
Rows touched: 179
sched_dep_time / sched_arr_time == 2400: 0 0
```

Decision 5 — Cancelled flights (no departure time): flag, do not drop

In [14]:
```
df["cancelled"] = df["dep_time"].isna()
print("Cancelled:", int(df["cancelled"].sum()), f"({df['cancelled'].mean():.2%})")
print("All delay/air-time values missing for these rows:",
      bool(df.loc[df["cancelled"], ["dep_delay", "arr_time", "arr_delay", "air_time"]].isna().all().al
l()))
```

```
Cancelled: 8255 (2.45%)
All delay/air-time values missing for these rows: True
```

Decision 6 — Missing `tailnum`: leave as missing

In [15]:
```
miss_tail = df["tailnum"].isna()
print("Missing tailnum:", int(miss_tail.sum()), f"({miss_tail.mean():.2%})")
print("...of which cancelled:", int((miss_tail & df["cancelled"]).sum()))
print("...of which flew:", int((miss_tail & ~df["cancelled"]).sum()))
```

```
Missing tailnum: 2512 (0.75%)
...of which cancelled: 2512
...of which flew: 0
```

Decision 7 — Flew but no arrival delay / air time (probably diverted): flag, do not drop

In [16]:
```
df["no_arrival_delay"] = df["arr_delay"].isna() & ~df["cancelled"]
print("Flagged:", int(df["no_arrival_delay"].sum()), f"({df['no_arrival_delay'].mean():.2%})")
print("  has arr_time but no arr_delay:", int((df["no_arrival_delay"] & df["arr_time"].notna()).sum())
)
print("  has no arr_time:", int((df["no_arrival_delay"] & df["arr_time"].isna()).sum()))
print("Reconciliation of arr_delay missing:", int(df["arr_delay"].isna().sum()), "=",
      int(df["cancelled"].sum()), "+", int(df["no_arrival_delay"].sum()))
```

```
Flagged: 1175 (0.35%)
  has arr_time but no arr_delay: 717
  has no arr_time: 458
Reconciliation of arr_delay missing: 9430 = 8255 + 1175
```

Decision 8 — Extreme and negative delays: keep

In [17]:
```
dd = df["dep_delay"].astype("float")
print("dep_delay >= 180 min:", int((dd >= 180).sum()), "   >= 720 min:", int((dd >= 720).sum()), "   m
ax:", dd.max())
# is the delay consistent with the clock times (modulo 24 h)? if so it is real, not a typing error
to_min = lambda x: (x // 100) * 60 + x % 100
gap = ((to_min(df["dep_time"].astype("float")) - to_min(df["sched_dep_time"]) - dd) % 1440).dropna()
print("dep_delay inconsistent with clock times:", int((gap != 0).sum()))
print("Early departures (dep_delay < 0):", int((dd < 0).sum()), "   early arrivals:", int((df["arr_del
ay"].astype("float") < 0).sum()))
```

```
dep_delay >= 180 min: 3945    >= 720 min: 29    max: 1301.0
dep_delay inconsistent with clock times: 0
Early departures (dep_delay < 0): 183575    early arrivals: 188933
```

Decision 9 — Duplicates

In [18]:
```
key = ["time_hour", "carrier", "flight", "origin", "dest", "sched_dep_time"]
print("Exact duplicate rows:", int(df.duplicated().sum()))
print("Duplicates on flight key:", int(df.duplicated(key).sum()))
```

```
Exact duplicate rows: 0
Duplicates on flight key: 0
```

Decision 10 — Same route, different distance

In [19]:
```
g = df.groupby(["origin", "dest"])["distance"]
bad = g.nunique()[lambda s: s > 1].index
print("Routes with more than one distance:", len(bad))
mode = g.transform(lambda s: s.mode().iloc[0])
off = (df["distance"] != mode)
print("Rows that differ from the route's usual distance:", int(off.sum()), "  max gap (miles):", int((
df["distance"] - mode).abs().max()))
print(df[df.set_index(["origin", "dest"]).index.isin(bad)].groupby(["origin", "dest"])["distance"].val
ue_counts())
```

```
Routes with more than one distance: 2
Rows that differ from the route's usual distance: 95   max gap (miles): 1
origin  dest  distance
EWR     EGE   1726        59
              1725        51
JFK     EGE   1747        59
              1746        44
Name: count, dtype: int64
```

Decision 11 — Implausible ground speed (distance / air_time)

In [20]:
```
mph = df["distance"] / (df["air_time"].astype("float") / 60)
print("Ground speed > 650 mph:", int((mph > 650).sum()), "   max:", round(mph.max(), 1))
print(df.loc[mph > 650, ["origin", "dest", "distance", "air_time"]])
print("Ground speed < 150 mph:", int((mph < 150).sum()), "(all short hops?)  longest distance among th
em:", int(df.loc[mph < 150, "distance"].max()))
```

```
Ground speed > 650 mph: 2    max: 703.4
       origin dest  distance  air_time
216447    LGA  ATL       762        65
251999    EWR  MSP      1008        93
Ground speed < 150 mph: 352 (all short hops?)  longest distance among them: 229
```

Decision 12 — Categorical codes

In [21]:
```
for c in ["carrier", "origin", "dest", "tailnum", "name"]:
    s = df[c].dropna()
    print(c, "| distinct:", s.nunique(), "| leading/trailing spaces:", int((s != s.str.strip()).sum())
,
          "| not upper-case:", int((s != s.str.upper()).sum()) if c != "name" else "n/a")
print("Constant columns:", [c for c in df.columns if df[c].nunique(dropna=False) == 1])
```

```
carrier | distinct: 16 | leading/trailing spaces: 0 | not upper-case: 0
origin | distinct: 3 | leading/trailing spaces: 0 | not upper-case: 0
dest | distinct: 105 | leading/trailing spaces: 0 | not upper-case: 0
tailnum | distinct: 4043 | leading/trailing spaces: 0 | not upper-case: 0
name | distinct: 16 | leading/trailing spaces: 0 | not upper-case: n/a
Constant columns: ['year']
```

Result

In [22]:
```
print(f"Rows: {len(df):,}   Columns: {df.shape[1]}   (working file had 20; +4 derived/flag columns)")
analysis_dep = df[~df["cancelled"]]                          # use for dep_delay questions
analysis_arr = df[~df["cancelled"] & ~df["no_arrival_delay"]]   # use for arr_delay / air_time questio
ns
print("dep_delay analysis rows:", f"{len(analysis_dep):,}", "  arr_delay analysis rows:", f"{len(analy
sis_arr):,}")
```

```
Rows: 336,776   Columns: 24   (working file had 20; +4 derived/flag columns)
dep_delay analysis rows: 328,521   arr_delay analysis rows: 327,346
```